YaniMed

System Design & Architecture

AI-powered PDF-grounded medical reviewer with integrated Mock Test module

DOCUMENT STATUS	SCOPE
Proposed design blueprint; implementation requires repository verification	Shared Subjects and PDFs, ingestion/RAG, Mock Test, security, database, workflows and delivery phases

Design principles

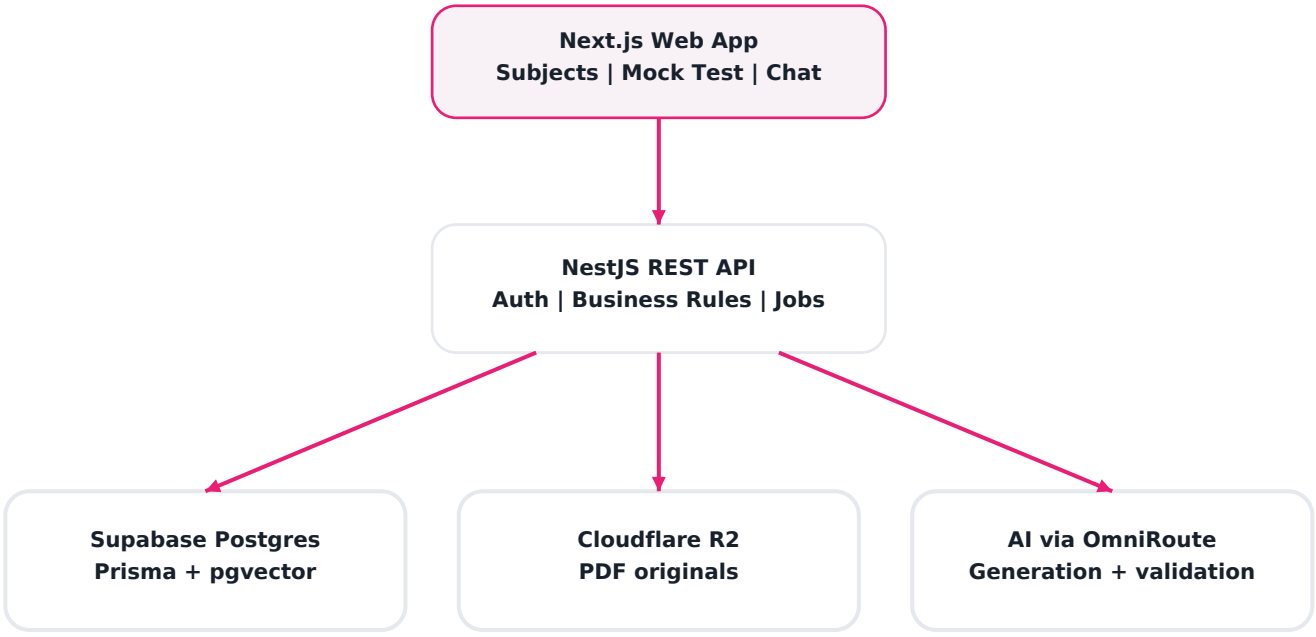
- One shared Subject entity across Subjects and Mock Test.
- Only uploaded and successfully processed PDFs ground medical answers and exam questions.
- Exactly 50 validated questions per playable mock test.
- Durable, recoverable jobs and safe offline exam progress.
- Preserve existing YaniMed features and implement in verified phases.

Document note

This document records the agreed target architecture and behavioral requirements. It is not an audit of the current repository, nor a claim that the components have already been implemented.

1. Executive architecture

YaniMed is a PDF-grounded study application. Users organize reviewer PDFs by subject, ask citation-backed questions, and take subject-specific mock tests. The web client communicates with a NestJS backend that owns authorization, persistence, retrieval, generation, and grading.



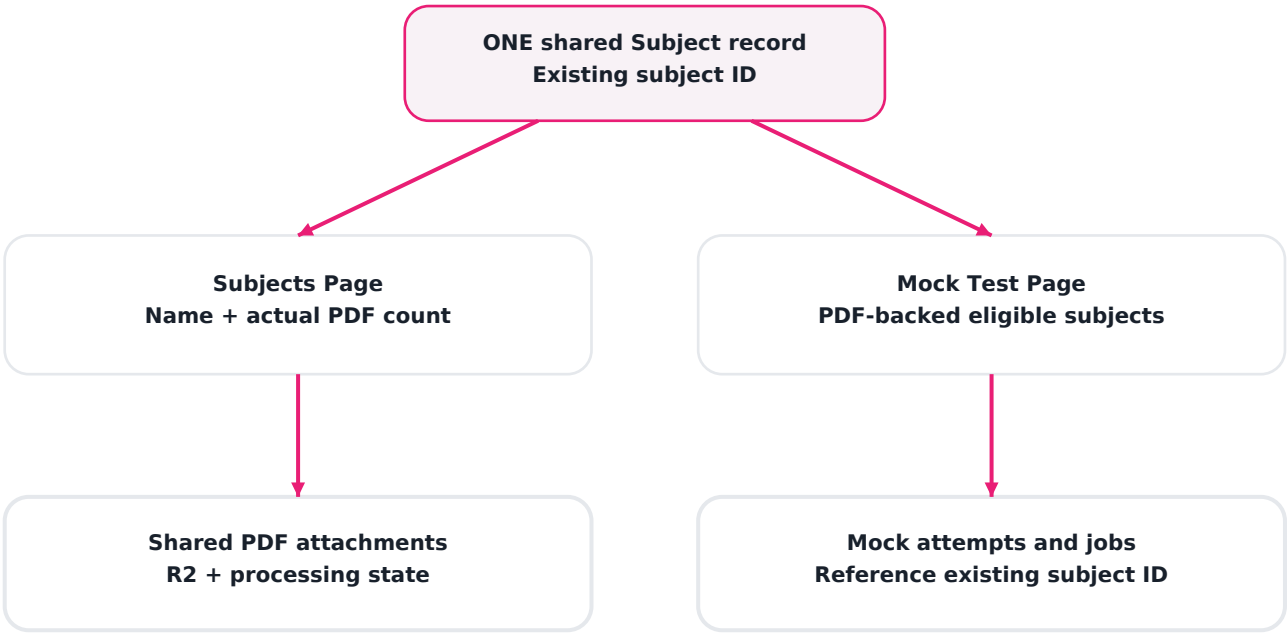
Proposed logical architecture - exact integrations require repository verification

Core technology decisions

Layer	Planned technology / responsibility
Client	Next.js, TypeScript, Tailwind CSS, shadcn/ui; TanStack Query and Zustand as appropriate
API and workers	NestJS, validated DTOs, durable background job processing
Database	Supabase PostgreSQL, Prisma ORM, pgvector for retrieval
File storage	Cloudflare R2 for original PDFs; authenticated PDF viewing
PDF ingestion	Text extraction with physical page mapping; OCR fallback for scanned pages
AI	Gemini embeddings; existing OmniRoute integration for AI provider routing and bounded retries

2. Shared Subjects architecture

Subjects and Mock Test are two views of the same domain data. Creating Pediatrics in Mock Test must create one subject record that also appears in Subjects. Attaching two PDFs must show “2 PDFs” on the Subjects page, not “50 items”.



No duplicate subject table and no duplicate PDF objects

Display and synchronization rules

- Subjects page: display every owned subject, including those with zero PDFs; show name and PDF count only.
- Mock Test dashboard: display subjects with at least one attached PDF; generation requires sufficient processed content.
- Create, rename, attach, delete and PDF count changes use the same subject ID and attachment records.
- Invalidate/refetch relevant client queries after mutations; never duplicate R2 objects solely for Mock Test.
- Entire-subject deletion requires confirmation and authorized cleanup of associated attempts and files.

3. PDF ingestion and retrieval

PDFs are uploaded once and processed into page-aware searchable content. Chat retrieval and Mock Test generation may share this indexed source, but must use different output policies.

Stage	Expected behavior
Upload	Authorize subject ownership, store original in R2, create durable document record
Extract	Extract text by actual physical PDF page; OCR pages with inadequate extractable text
Chunk	Store source document ID, actual PDF page, optional verified printed page/chapter
Embed/index	Generate embeddings and index in pgvector; manage rate limits and retries
Ready	Mark PDF ready only after processing and index integrity checks
Retrieve	Scope strictly to the current user and selected subject; never use web fallback

Citation invariant

Every cited question must reference a real source document and actual PDF page. Printed book pages and chapters are optional unless reliably verified. Deleted individual PDFs retain historical citation metadata and display “Source Unavailable”.

4. Mock Test domain model

The following is a conceptual schema, not a mandate to duplicate models already present in Prisma.

Entity	Purpose / key relationships
Subject (existing)	Single canonical subject; owned by user
PDF / Document (existing)	Attached to subject; original storage key, processing status, page metadata
MockAttempt	User + subject; lifecycle and persisted question/choice order
GenerationJob	Attempt/subject, progress 0-50, state, cancellation and recovery checkpoints
MockQuestion / Choice	Immutable generated vignette, topic, explanations, answer key and choice IDs
QuestionCitation	Document ID, actual PDF page, verified optional printed page/chapter
AttemptAnswer	One answer per question; revisions, sync/idempotency metadata
AttemptSession	Current editing-session lease/token and takeover version
AttemptResult	Immutable grade snapshot and per-topic performance
QuestionReport	User, attempt, question, reason, optional details, review status

Critical constraints

- One unfinished attempt per user and subject; concurrent requests must be transaction-safe.
- Exactly 50 validated questions before READY; never publish partial question sets.
- Question and answer-choice order are generated once, persisted, and never reshuffled.
- Correct answer keys are never sent in exam-safe API responses.
- Submitted scores and answer snapshots are immutable and graded server-side.

5. Question-generation pipeline

Start → retrieve processed subject content → draft clinical questions → structural and source-grounded validation → persist accepted questions → repeat until 50 → full-set validation → READY.

Question specification

Each item contains a five-sentence clinical vignette, clear lead-in, four plausible choices, one best answer, educational objective, explanation for the correct choice and each distractor, primary medical topic, and verified PDF citations.

Durability and safety

- Background job must survive page navigation and support checkpoint-based recovery.
- Progress reflects accepted, validated questions, not attempted AI calls.

- Reject clinically questionable, ambiguous, unsupported, or substantially duplicate questions.
- Use existing OmniRoute provider routing with bounded retries, backoff and explicit failure states.
- Cancel Generation requires confirmation, persists cancellation, discards incomplete job content and prevents late publication.
- Insufficient content blocks release; automated validation reduces but cannot eliminate medical errors.

6. Exam state machine

State	Transitions and UI behavior
GENERATING	Background progress; cancel or recover failed job
READY	Show Start Test; no automatic navigation
IN_PROGRESS	Forward-only Answer & Continue or Skip; Back exits to dashboard
AWAITING_SUBMISSION	All 50 answered; editable neutral navigator; explicit Submit
COMPLETED	Immutable grade, Results, View Answers, Retry, Done
FAILED / CANCELLED	Meaningful status; safe retry/new generation

Forward-only navigation

First pass traverses Questions 1–50 in the persisted shuffled order. Skip leaves a question unanswered. After Question 50, revisit unanswered questions in ascending displayed number; repeat until all are answered. No Previous or number navigator during active exam.

Pre-submission review

All 50 numbers are neutral; only the selected number is highlighted. Students may jump, use Previous/Next, edit choices and autosave. No answer keys, explanations, citations, or correctness indicators appear until submission.

Offline and session ownership

- Save answers locally during disconnection; allow progress only through already-loaded questions.
- Synchronize when reconnected using revision-safe, idempotent operations; block submission until fully synced.
- One active editing session per attempt; explicit takeover revokes the prior session’s editing authority.
- Old session submissions, saves and offline synchronization must be rejected, not silently merged.
- Submission is atomic and idempotent; all 50 answers are required.

7. Results, review and reports

Grade each question as 1 correct or 0 incorrect; score is /50, with percentage and correct/incorrect counts. Do not show pass/fail or an exam timer. Results include topic breakdowns and Retry, View Answers, Done.

Post-exam review shows original shuffled choices, selected and correct answers, explanations, citations, and All/Correct/Incorrect filters combinable with a topic filter. Question reports can flag answer keys, ambiguity, citations, explanations or other issues without changing scores automatically.

8. API and trust boundaries

API capability	Security / consistency rule
Subject and PDF operations	Authenticated ownership; reuse existing routes and storage workflow
Generation start/status/cancel	Idempotency, job ownership, race-safe state transitions
Exam retrieval	Exam-safe DTO: no keys, correctness, explanations or hidden citations
Answer/save/sync	Session lease validation, monotonic revision checks, server ownership
Submit	Server-only grading, complete-answer check, atomic transaction
Results/history/review	Only completed attempts; immutable answer mapping and citation metadata
Question reports	Authenticated reporter and attempt/question ownership

Security requirements

- Treat uploaded PDF text and model outputs as untrusted; prevent prompt injection and secret disclosure.
- Use signed/authenticated access to PDFs rather than public unrestricted R2 object links.
- Keep API keys and provider routing on the server.
- Apply DTO validation, authorization, rate limits where appropriate, and audit-friendly error logging.
- Use additive reviewed migrations; do not reset the database or alter unrelated application behavior.

9. Deployment and reliability

Use the existing deployment topology where possible. Generation requires a durable queue/worker and persistent job state; an HTTP request or browser tab must not be the sole execution context. Monitor failed jobs, provider rate limits, processing lag, stale session leases, and synchronization conflicts. Configure bounded retries and operational recovery paths.

10. Phased delivery and acceptance

Phase	Exit criterion
1. Repository audit	Existing services, schema, tests and risks documented; baseline verified
2. Data and API foundation	Additive migrations; ownership, concurrency and lifecycle tests pass
3. Generation	Exactly 50 validated source-grounded questions; cancel/retry/recovery tested
4. Exam workflow	Shuffle, forward-only skips, autosave, offline sync, takeover verified
5. Submission and results	Neutral pre-review, atomic grading, topic analytics and history verified
6. Citations and reports	Authenticated viewer, deleted-source behavior, reports and filters tested
7. Regression	Frontend/backend builds, integration and existing chat/PDF flows pass

11. Final acceptance checklist

- Pediatrics created in Mock Test appears in Subjects; two attached PDFs show “2 PDFs”, never “50 items”.
- Only processed, PDF-grounded material is used for generation; all 50 questions pass validation.
- Question and choice order remain stable across refresh and resume.
- Skip/revisit and pre-submission editing follow the agreed navigation rules.
- Offline changes synchronize safely and stale sessions cannot overwrite progress.
- Results, citations, reports and historical attempts remain consistent across PDF changes.
- Existing Subjects, chat, authentication and upload workflows pass regression tests.

Implementation note

Before coding, reconcile this blueprint with the real YaniMed repository. Where current implementation differs, propose the smallest safe migration path and verify each phase before proceeding.